
SOFTWARE REQUIREMENTS SPECIFICATION

SISTEM DE APLICARE A FILTRELOR UNEI IMAGINI ÎN PARALEL

Prepared by:

Băloi Raul Adrian Theodor
Bîrdeanu Aaron-Sasha
Cotrău Darius Andrei
Bărbascu Raul Andrei

Cuprins

1	Introducere	3
1.1	Scopul documentului	3
1.2	Public țintă	3
1.3	Scopul proiectului	3
1.4	Arhitectura sistemului	3
1.5	Clase de utilizatori și caracteristici	3
1.6	Funcțiile produsului	4
1.7	Mediul de operare	4
1.8	Designul sistemului	4
2	Funcționalități ale sistemului	6
2.1	Descrieri și priorități	6
2.2	Cerințe funcționale	6
2.2.1	Cerințe server	6
2.2.2	Cerințe client	6
3	Alte cerințe nefuncționale	7
3.1	Cerințe de performanță	7
3.2	Cerințe de securitate	7
3.3	Atribute de calitate software	7

1 Introducere

1.1 Scopul documentului

Scopul acestui document este de a descrie cerințele unui sistem distribuit de procesare a imaginilor, care aplică filtre folosind tehnici de procesare paralelă. Sistemul este proiectat pentru a procesa imaginile eficient prin împărțirea acestora în patru regiuni și procesarea simultană folosind mai multe procese.

1.2 Public țintă

Documentul este destinat coordonatorilor de proiect și echipei care dezvoltă sistemul și dezvoltatorilor C care pot oferi o critică constructivă asupra proiectului.

1.3 Scopul proiectului

Sistemul propus implementează o arhitectură client-server pentru aplicarea simplă de filtre asupra imaginilor. Aplicarea filtrelor în mod paralel și monitorizarea clienților și a serverului.

1.4 Arhitectura sistemului

- Server: C + gSOAP + GraphicsMagick, cu o coadă FIFO internă de joburi procesată de un thread worker dedicat
- Client TCP: C, protocol binar propriu (fără gSOAP), pentru transfer eficient de imagini mari
- Client Web: HTML/CSS/JavaScript, comunicare SOAP 1.1 peste HTTP (fără gSOAP pe partea de client)
- Interfață admin: C + ncurses

1.5 Clase de utilizatori și caracteristici

- Utilizatorul final:
 - Încarcă imaginea
 - Selectează filtrul
 - Primește imaginea procesată
- Administratorul:

- Monitorizează clienții conectați și job-urile acestora
- Vizualizează o statistică a filtrelor
- Configurează setări ale serverului
- Elimină clienți
- Vizualizează resursele utilizate de către server
- Vizualizează log-urile

1.6 Funcțiile produsului

- API SOAP pentru încărcarea și procesarea imaginilor (sincron — applyFilter — și asincron, pe bază de tichete — submitJob/jobStatus)
- Server TCP raw, cu protocol binar propriu, pentru transferul eficient de imagini mari
- Coadă FIFO de joburi cu tichete, procesate asincron de un thread worker
- Procesare paralelă a imaginilor folosind fork()
- Filtrarea imaginilor folosind GraphicsMagick
- Transfer de imagini în format binar
- Administrare prin interfața terminalului

1.7 Mediul de operare

- Linux

1.8 Designul sistemului

Fluxul de procesare:

1. Clientul (TCP sau Web/SOAP) trimite imaginea și filtrul către server
2. Serverul preia imaginea din payload-ul binar și o pune într-o coadă FIFO de joburi, răspunzând imediat cu un tichet
3. Un thread worker dedicat ridică jobul cel mai vechi din coadă (FIFO) și apelează procesarea
4. Imaginea este împărțită în regiuni
5. Fiecare regiune este procesată într-un proces copil
6. Rezultatele sunt colectate și combinate
7. Clientul interoghează periodic tichetul, iar imaginea finală este trimisă clientului la finalizare

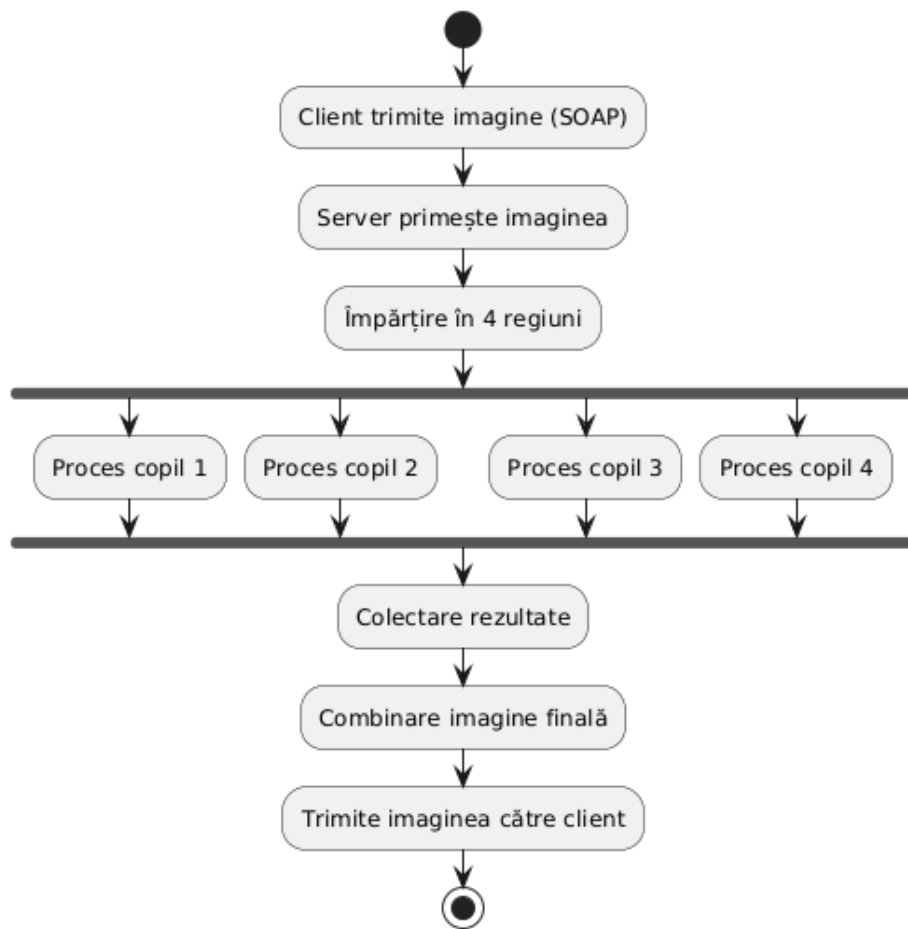


Figura 1.1: Fluxul de procesare al sistemului

2 Funcționalități ale sistemului

2.1 Descreri și priorități

1. Procesare imagini (High)
2. Execuție paralelă (High)
3. Comunicare SOAP (High)
4. Comunicare TCP raw pentru fișiere mari (High)
5. Coadă FIFO de joburi asincrone (High)
6. Monitorizare admin (Medium)
7. Interfață client (Medium)

2.2 Cerințe funcționale

2.2.1 Cerințe server

- Implementarea serviciilor SOAP folosind gSOAP (connect, echo, applyFilter, submitJob, jobStatus, bye, serverInfo)
- Acceptarea imaginilor în format binar, atât prin SOAP cât și prin protocolul TCP raw dedicat
- Gestionarea unei cozi FIFO de joburi cu tichete, procesate de un thread worker în ordinea sosirii
- Aplicarea filtrelor folosind librăria GraphicsMagick
- Utilizarea fork() pentru paralelizare
- Sincronizarea accesului la coada de joburi și la starea globală prin mutex-uri și variabile de condiție

2.2.2 Cerințe client

- Conectarea la server și obținerea unui identificator de client
- Încărcarea imaginilor (prin SOAP, codificate base64, sau direct, prin protocolul TCP)
- Trimiterea tipului de filtru
- Trimiterea jobului, obținerea unui tichet și interogarea periodică a stării jobului
- Primirea imaginii procesate și a timpului de procesare

3 Alte cerințe nefuncționale

3.1 Cerințe de performanță

- Procesarea paralelă trebuie să reducă timpul de execuție comparativ cu procesarea secvențială
- Sistemul trebuie să suporte imagini de dimensiuni medii
- Coada FIFO nu trebuie să blocheze operațiile de submit/poll ale altor clienți pe durata procesării unei imagini
- Clientul TCP trebuie să permită transferul eficient al imaginilor mari, fără overhead de codificare base64/SOAP

3.2 Cerințe de securitate

- Restricționarea accesului la interfața de administrator
- Respingerea conexiunilor de la adrese IP banate de administrator

3.3 Atribute de calitate software

- Fiabilitate: procesare stabilă fără erori
- Mentenabilitate: cod modular în C
- Concurență sigură: accesul la starea globală și la coada de joburi este protejat prin mutex-uri și variabile de condiție